

SupportIQ

Complete Build & Submission Guide

DOTMappers IT Pvt. Ltd. — AI Engineer Assessment

Objective: Build an AI-powered customer support ticket intelligence system that ingests CSV data, answers natural-language questions using an LLM, detects anomalies, and exposes the functionality through a REST API and a minimal UI.

Assessment constraint	Implementation decision
Language	Python only
LLM	Groq Free Plan — configurable model
Backend	FastAPI
Frontend	Streamlit + Plotly
Data	Pandas + SQLite-ready service layer
Delivery	GitHub repository + ZIP + README
Target	Functional, explainable, locally runnable prototype

Important: Groq's Free Plan is intended to be free within its limits. Groq states that users on the Free Plan are not charged; exceeding limits results in rate-limit errors. Limits can change, so verify the limits displayed in your own Groq Console and do not upgrade to a paid Developer plan for this assessment.

Prepared for Harish Robin H • 24 September 2026

1. Executive Summary

You should build a focused, reliable prototype rather than an overcomplicated platform. The evaluator is explicitly scoring functionality, architecture, code quality, LLM integration, and documentation. Your strongest strategy is to make the LLM interpret questions while Python performs all calculations deterministically.

The project name used throughout this guide is **SupportIQ — AI-Powered Customer Support Intelligence Platform**.

The four required capabilities are:

- 1 Ingest the supplied support_tickets.csv file and validate its schema.
- 2 Answer natural-language questions such as counts, averages, grouping, filtering, and comparisons.
- 3 Detect and explain operational anomalies such as unresolved critical tickets and unusually long resolution times.
- 4 Expose the system through FastAPI and a minimal Streamlit interface.

Recommended system behavior:

- The user submits a natural-language question.
- Groq converts the question into a strict JSON query plan.
- Pydantic validates the plan against an allow-list of operations and fields.
- Python executes the plan against the dataset.
- The system returns structured results and, optionally, asks Groq to phrase the verified result in natural language.
- The UI displays the answer, filters, and supporting records so the result is auditable.

2. Assessment Requirements — Source of Truth

The supplied assessment document states that the system must use Python, an LLM, a REST API and/or minimal UI covering the required functionality, a README, and a requirements.txt or equivalent. It also requires the system to start with a single command and be runnable at zero cost.

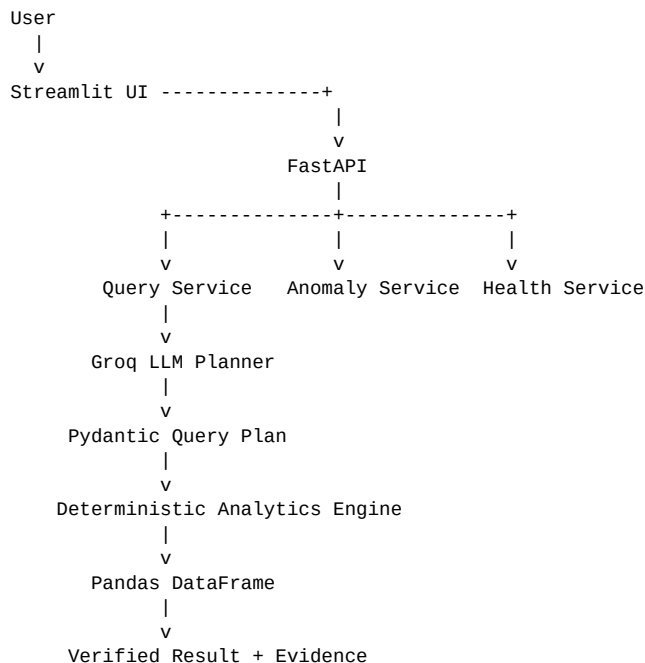
Requirement	What you must deliver	How SupportIQ satisfies it
CSV ingestion	Load and validate support_tickets.csv	Data loader with schema checks and clean DataFrame
Natural-language querying	Questions about the dataset	Groq JSON query planner + deterministic analytics engine
Anomaly detection	Flag unusual or risky tickets	Business rules + IQR statistical detection
REST API / UI	At least three endpoints or equivalent UI	FastAPI endpoints plus Streamlit dashboard
LLM use	LLM for natural-language understanding	Groq structured-output prompt
Documentation	Setup, architecture, model, examples, limitations	Detailed README and architecture diagram
Zero-cost execution	No paid services required	Groq Free Plan + local Python stack
Single-command startup	Evaluator must start easily	run.sh and documented commands

The assessment includes sample questions such as: How many tickets are currently open? Which agent resolved the most tickets this month? Show all Critical tickets not resolved within 12 hours. What is the average customer rating for Technical tickets? Are there anomalies in resolution times this week?

Dataset naming note: use the actual CSV column names as the source of truth. The document's sample schema uses abbreviated names in one place, while the detailed descriptions use response_time_hrs, resolution_time_hrs, and customer_rating. Your code should support the actual CSV names.

3. Recommended Architecture

Use a layered architecture. This makes the project easier to test, explain, and extend during the 30-minute walkthrough.



Layer responsibilities:

Layer	Responsibility
Data layer	Load CSV, normalize types, validate required columns, expose clean DataFrame
Analytics layer	Count, filter, group, average, min/max, ranking, date filtering
LLM layer	Translate natural language into a structured query plan; never calculate raw metrics
Anomaly layer	Apply transparent business rules and statistical thresholds
API layer	Validate requests, call services, return consistent JSON responses
UI layer	Show KPIs, query interface, anomaly table, ticket explorer, system status
Documentation	Explain trade-offs, setup, examples, limitations, and future improvements

Why not let the LLM directly write arbitrary Python or SQL? Because unrestricted code generation introduces security, reproducibility, and hallucination risks. A constrained query plan is easier to validate and defend.

4. Project Folder Structure

```
supportiq/
├── app/
│   ├── __init__.py
│   ├── main.py                # FastAPI entry point
│   ├── config.py              # Environment settings
│   ├── schemas.py             # Pydantic request/response models
│   ├── dependencies.py
│   ├── services/
│   │   ├── data_service.py    # CSV loading and validation
│   │   ├── analytics_service.py
│   │   ├── llm_service.py     # Groq integration
│   │   ├── query_service.py   # Plan validation + execution
│   │   └── anomaly_service.py
│   ├── utils/
│   │   ├── logging_config.py
│   │   └── date_utils.py
│   └── data/
│       └── support_tickets.csv
├── frontend/
│   └── streamlit_app.py
├── tests/
│   ├── test_data_service.py
│   ├── test_analytics.py
│   ├── test_anomalies.py
│   └── test_api.py
├── docs/
│   └── architecture.md
├── .env.example
├── .gitignore
├── requirements.txt
├── README.md
├── run.sh
└── Dockerfile                # Optional, only if time permits
```

Keep the implementation modular, but do not create unnecessary abstractions. A small assessment benefits from readable services and straightforward tests.

5. Groq Setup — Free and Fast

Use Groq as the hosted LLM provider. Groq documents a Free Plan, and its community FAQ states that users are not charged while remaining on the Free Plan; requests that exceed free limits receive rate-limit errors rather than creating a charge. Groq's official model documentation lists OpenAI GPT-OSS models and their performance characteristics. Always confirm the currently available model IDs and limits in your own console.

Recommended configuration:

Setting	Value
Provider	Groq Cloud
Model variable	GROQ_MODEL=openai/gpt-oss-20b or another currently available free model
API key variable	GROQ_API_KEY
SDK	groq
Temperature	0 or as low as supported for predictable query planning
Output format	Strict JSON validated by Pydantic
Fallback	Return a clear error if the API key is missing or rate-limited

Create a key:

- 1 Open <https://console.groq.com/>.
- 2 Create or sign in to your account.
- 3 Create an API key.
- 4 Store it only in a local .env file.
- 5 Never commit .env, API keys, or secrets to GitHub.
- 6 Keep the model configurable because model availability and free limits can change.

```
# .env.example
GROQ_API_KEY=replace_with_your_key
GROQ_MODEL=openai/gpt-oss-20b
DATA_PATH=data/support_tickets.csv
APP_ENV=development

# Install dependencies
pip install fastapi uvicorn streamlit pandas numpy plotly pydantic-settings python-dotenv groq httpx pytest
```

Free-tier safety checklist:

- Do not upgrade to a paid plan for this assessment.
- Do not put the API key in the frontend code.
- Do not send the full CSV to Groq for every question.
- Send only the question, allowed schema, and compact instructions.
- Cache or reuse the loaded DataFrame locally.
- Handle HTTP 429 errors gracefully.

6. Feature-by-Feature Build Plan

Build in the following order. Do not begin with UI polish. Each feature should work locally before moving to the next.

Feature 1 — Data ingestion and validation

Goal: load the CSV once at application startup and validate that required columns exist.

```
REQUIRED_COLUMNS = [
    "ticket_id", "created_at", "category", "priority", "status",
    "response_time_hrs", "resolution_time_hrs", "agent_id",
    "customer_rating", "issue_summary"
]

df = pd.read_csv(DATA_PATH)
missing = set(REQUIRED_COLUMNS) - set(df.columns)
if missing:
    raise ValueError(f"Missing required columns: {sorted(missing)}")

df["created_at"] = pd.to_datetime(df["created_at"], errors="coerce")
df["response_time_hrs"] = pd.to_numeric(df["response_time_hrs"], errors="coerce")
df["resolution_time_hrs"] = pd.to_numeric(df["resolution_time_hrs"], errors="coerce")
df["customer_rating"] = pd.to_numeric(df["customer_rating"], errors="coerce")
```

Acceptance criteria:

- Application refuses to start with a clear error if the file is missing.
- Invalid dates and numeric fields are reported.
- Null resolution times are preserved for unresolved tickets.
- A health endpoint reports the row count and data status.

Feature 2 — Deterministic analytics engine

Implement reusable functions rather than writing separate one-off code for each question.

Function	Example
count_rows(filters)	How many tickets are open?
aggregate(metric, group_by, filters)	Average rating by category
rank_agents(metric, direction, filters)	Agent with most resolved tickets
filter_tickets(filters)	Critical tickets unresolved over 12 hours
summary_metrics()	Total, open, resolved, escalated, critical

Supported operations should initially include:

- count
- average
- sum
- min
- max
- group_count
- group_average
- top_n
- filter
- list_records

Do not attempt to support every possible English question. Support a well-defined set of analytical capabilities and document the supported scope.

7. LLM Query Planning

This is the most important AI engineering feature. The LLM should translate the question into a controlled JSON plan.

Suggested Pydantic schema:

```
class QueryPlan(BaseModel):
    operation: Literal[
        "count", "average", "sum", "min", "max",
        "group_count", "group_average", "top_n",
        "filter", "list_records"
    ]
    metric: str | None = None
    group_by: str | None = None
    filters: dict[str, Any] = {}
    limit: int = 20
    explanation: str | None = None
```

Allowed fields:

```
ALLOWED_FIELDS = {
    "ticket_id", "created_at", "category", "priority", "status",
    "response_time_hrs", "resolution_time_hrs", "agent_id",
    "customer_rating", "issue_summary"
}

ALLOWED_FILTER_VALUES = {
    "category": {"Billing", "Technical", "General"},
    "priority": {"Low", "Medium", "High", "Critical"},
    "status": {"Open", "Resolved", "Escalated"}
}
```

System prompt for Groq:

You are a query-planning assistant for a customer support analytics dataset.

Your job is to convert a user question into one valid JSON query plan.
Do not calculate results. Do not write Python. Do not invent fields.
Use only the allowed operations and fields.
If the question cannot be answered from the dataset, return:
{
 "operation": "unsupported",
 "explanation": "..."
}

Dataset fields:
ticket_id, created_at, category, priority, status,
response_time_hrs, resolution_time_hrs, agent_id,
customer_rating, issue_summary

Allowed operations:
count, average, sum, min, max, group_count,
group_average, top_n, filter, list_records

Return JSON only.

Example:

User: How many critical tickets are unresolved?

```
Plan:
{
  "operation": "count",
  "filters": {
    "priority": "Critical",
    "status": ["Open", "Escalated"]
  },
  "limit": 20
}
```

Validation rules:

- Reject unknown operations.
- Reject unknown fields.

- Validate category, priority, and status values.
- Cap result limits, for example at 100.
- Do not execute arbitrary code returned by the LLM.
- If the plan is invalid, retry once with a correction prompt or return a useful error.

8. Anomaly Detection Engine

Use explainable rules first. The evaluator should be able to understand exactly why a ticket was flagged.

Rule	Detection logic	Explanation shown to user
Critical unresolved	Priority = Critical and status != Resolved	Critical ticket remains unresolved
High-priority aging	Priority in High/Critical and unresolved age > 24 hours	High-priority ticket has exceeded 24-hour aging threshold
Long resolution	Resolved ticket resolution time > Q3 + 1.5 × IQR	Resolution time is statistically high compared with the dataset
Poor satisfaction	Resolved ticket customer_rating <= 2	Low customer satisfaction rating
Slow response	response_time_hrs > configurable percentile or threshold	First response time is unusually slow

IQR formula:

```
Q1 = df["resolution_time_hrs"].quantile(0.25)
Q3 = df["resolution_time_hrs"].quantile(0.75)
IQR = Q3 - Q1
upper_bound = Q3 + 1.5 * IQR
anomalies = df[df["resolution_time_hrs"] > upper_bound]
```

Recommended anomaly response shape:

```
{
  "ticket_id": "TKT-002",
  "anomaly_type": "critical_unresolved",
  "severity": "high",
  "reason": "Critical ticket is not resolved",
  "detected_value": null,
  "threshold": null
}
```

Avoid claiming that statistical anomalies are automatically fraud, failure, or root cause. Use neutral language such as *flagged for review*.

9. FastAPI Contract

Implement at least the following endpoints. Add /summary because it improves the UI and demonstrates system design.

Method	Endpoint	Purpose
GET	/health	Verify API, dataset, and LLM configuration status
GET	/summary	Return total tickets, status counts, priority counts, averages
POST	/query	Convert natural-language question into plan and execute it
GET	/anomalies	Return anomaly records with filters
GET	/tickets	Search and filter ticket records

Example request:

```
POST /query
{
  "question": "What is the average customer rating for Technical tickets?"
}
```

Example response:

```
{
  "question": "What is the average customer rating for Technical tickets?",
  "query_plan": {
    "operation": "average",
    "metric": "customer_rating",
    "filters": {"category": "Technical"}
  },
  "result": {
    "value": 3.74,
    "record_count": 162
  },
  "answer": "The average customer rating for Technical tickets is 3.74."
}
```

API error handling:

- 400 for invalid request payloads.
- 422 for schema validation failures.
- 503 when Groq is unavailable or the API key is missing.
- 500 only for unexpected server errors, with safe logs.
- Never expose the API key or stack trace in the response.

10. Streamlit UI Design

Build a clean, modern dashboard with a sidebar navigation or segmented navigation—not a crowded page full of radio buttons.

Pages / sections:

Section	Components
Overview	KPI cards, status donut, priority bar chart, category chart, recent alerts
Ask SupportIQ	Chat-style question input, answer card, query-plan expander, evidence table
Anomaly Center	Severity filters, anomaly count cards, sortable table, explanation column
Ticket Explorer	Search, category/priority/status filters, date range, ticket details
System Health	API status, dataset row count, Groq configuration status, last refresh

UI principles:

- Use consistent spacing and typography.
- Show loading states while calling the API.
- Display errors as helpful messages, not raw exceptions.
- Show the supporting records behind an answer.
- Use Plotly charts for interactive filtering.
- Keep the primary action obvious: Ask a question.
- Make the anomaly reason visible without requiring the user to inspect code.

Suggested color semantics: neutral base, blue for information, amber for review, red only for high-severity alerts, green for healthy/resolved states.

11. Antigravity IDE Prompts — Use in Sequence

Paste one prompt at a time into Antigravity. After each sprint, run the acceptance checks before continuing. Do not paste the entire project request repeatedly; incremental prompts reduce broken code and make debugging easier.

Prompt 1 — Scaffold the project

You are a senior Python AI engineer. Create a project named SupportIQ for the DOTMappers AI Engineer Assessment.

Requirements:

- Python only
- FastAPI backend
- Streamlit frontend
- Pandas for CSV analytics
- Groq API for LLM query planning
- Pydantic validation
- Modular services
- pytest tests
- .env.example and .gitignore
- README.md
- run.sh single-command startup helper

Create the folder structure exactly as specified in the project blueprint. Do not implement arbitrary code execution. Keep the initial scaffold runnable. After creating files, list the files created and commands to run the app.

Prompt 2 — Implement ingestion

Implement the data ingestion layer for SupportIQ.

Use data/support_tickets.csv as the source of truth. Validate these columns: ticket_id, created_at, category, priority, status, response_time_hrs, resolution_time_hrs, agent_id, customer_rating, issue_summary.

Requirements:

- clear error if file is missing
- parse created_at as datetime
- convert numeric fields safely
- preserve nulls for unresolved tickets
- expose row count and validation status
- add unit tests
- do not hardcode calculated metrics

Run the tests and report results.

Prompt 3 — Implement analytics

Implement a deterministic analytics service for SupportIQ.

Support operations: count, average, sum, min, max, group_count, group_average, top_n, filter, list_records.

Use allow-listed fields and safe filters. Support status Open, Resolved, Escalated; priority Low, Medium, High, Critical; category Billing, Technical, General.

Return structured results with record counts and supporting records where useful. Add tests for at least five realistic assessment questions. Do not use the LLM for calculations.

Prompt 4 — Implement Groq planner

Implement Groq integration for SupportIQ.

Use environment variables GROQ_API_KEY and GROQ_MODEL. Use the Groq Python SDK. Create a strict system prompt that converts natural-language questions into a JSON QueryPlan. Validate the response with Pydantic. Reject unknown operations and fields. Handle missing key, timeout, invalid JSON, and HTTP 429 errors. Never execute arbitrary code generated by the model.

Add a mock-based test so tests do not require a real API key.

Prompt 5 — Connect query service

Connect the Groq planner to the deterministic analytics engine.

Flow:

question -> Groq QueryPlan -> Pydantic validation -> analytics execution
-> verified result -> concise natural-language answer.

Return the query plan and evidence in the API response for transparency.
If the question is unsupported, explain the supported capabilities.
Add tests using mocked LLM plans.

Prompt 6 — Implement anomalies

Implement an explainable anomaly detection service.

Rules:

1. Critical unresolved tickets.
2. High/Critical unresolved tickets older than 24 hours.
3. Resolution time above $Q3 + 1.5 * IQR$.
4. Resolved tickets with customer rating ≤ 2 .
5. Slow response times using a documented threshold or percentile.

Return ticket_id, anomaly_type, severity, reason, detected_value, threshold.
Make thresholds configurable.
Add unit tests and ensure null resolution times are handled correctly.

Prompt 7 — Build FastAPI

Implement FastAPI endpoints:

GET /health
GET /summary
POST /query
GET /anomalies
GET /tickets

Use Pydantic request and response schemas.
Add clear error handling and OpenAPI descriptions.
Keep the API key server-side.
Test the endpoints with TestClient.

Prompt 8 — Build the UI

Build a modern Streamlit interface for SupportIQ.

Sections:

Overview, Ask SupportIQ, Anomaly Center, Ticket Explorer, System Health.

Use KPI cards, Plotly charts, clean spacing, readable tables,
loading states, empty states, and helpful errors.
Avoid an outdated radio-button-heavy layout.
The Ask SupportIQ page must show the answer, query plan, and evidence.
Connect to the FastAPI backend through a configurable API base URL.

Prompt 9 — Testing and hardening

Act as a strict evaluator for the DOTMappers AI Engineer Assessment.

Review the entire SupportIQ repository against:

- all four functional requirements
- Python-only constraint
- free-tier/no paid service constraint
- single-command startup
- modular architecture
- error handling
- LLM validation and prompt quality
- README completeness

Run tests, identify failures, fix them, and produce a final verification checklist.
Do not add unnecessary features that risk breaking the system.

12. Testing Checklist

Before submission, test from a clean terminal. Do not rely only on the UI working on your own machine.

Test	Expected result
CSV missing	Clear startup error explaining the missing file
CSV malformed	Validation error identifies missing/invalid columns
GET /health	200 response with dataset status
POST /query with valid question	Plan, result, answer, and evidence returned
POST /query with unsupported question	Helpful supported-scope response
Groq key missing	Graceful 503 or configuration message
Groq rate limit	Friendly retry/rate-limit message
Critical unresolved query	Correct count and supporting tickets
Average rating query	Null ratings excluded from average
Anomaly detection	No unresolved ticket incorrectly treated as resolved
UI API unavailable	Visible error state, not a crash
Fresh environment	Install and startup instructions work

Recommended sample queries:

- How many tickets are currently open?
- How many critical tickets are unresolved?
- What is the average customer rating for Technical tickets?
- Which agent resolved the most tickets?
- Show Critical tickets not resolved within 12 hours.
- What are the top three categories by ticket volume?
- List unresolved high-priority tickets.
- Are there anomalies in resolution times?

13. README.md Structure

Your README is worth marks. It should make the evaluator confident that the project is intentional and runnable.

- 1 Project title and one-paragraph overview.
- 2 Problem statement and business value.
- 3 Feature list.
- 4 Architecture diagram.
- 5 Technology stack.
- 6 Folder structure.
- 7 Prerequisites.
- 8 Installation steps.
- 9 Environment variables.
- 10 How to run the API.
- 11 How to run the Streamlit UI.
- 12 API endpoint table.
- 13 Supported natural-language query types.
- 14 Example questions with real outputs.
- 15 Anomaly rules and thresholds.
- 16 LLM prompt and validation approach.
- 17 Testing instructions and results.
- 18 Known limitations.
- 19 Future improvements.
- 20 Screenshots or GIF if available.

Known limitations you can honestly document:

- The query planner supports a defined set of analytical operations rather than unrestricted questions.
- The dataset is static and supplied as a CSV.
- Groq availability and free-tier limits are external dependencies.
- Anomaly rules are heuristics and flag records for human review.
- Date phrases such as 'this week' are interpreted relative to the dataset's date range unless explicitly configured.
- Authentication and multi-user access are outside the assessment scope.

Submission packaging:

- Push the complete repository to GitHub.
- Include the dataset if permitted by the assessment; otherwise include a clear data placement instruction.
- Remove .env and all secrets.
- Create a ZIP containing the completed project.
- Verify the ZIP opens correctly and includes README.md and requirements.txt.

- Email the GitHub link and ZIP to Vidyavathi.V@dotmappers.in.
- Use the requested subject format: [AI Engineer Assessment] — Your Name.

14. Time-Boxed Execution Plan

Time block	Work	Definition of done
Hour 0–1	Create repo, environment, scaffold	App folders, requirements, env example
Hour 1–3	Ingestion + schema validation	CSV loads, types clean, tests pass
Hour 3–6	Analytics engine	Sample questions work deterministically
Hour 6–9	Groq planner + validation	Question becomes safe QueryPlan
Hour 9–11	Query orchestration	End-to-end /query works
Hour 11–13	Anomaly engine	Rules, explanations, tests
Hour 13–15	FastAPI endpoints	Health, summary, query, anomalies
Hour 15–18	Streamlit UI	Overview, Ask, Anomalies, Explorer
Hour 18–20	Testing and bug fixes	Clean run, API tests, error handling
Hour 20–22	README and screenshots	Evaluator can reproduce setup
Hour 22–24	GitHub, ZIP, final verification	Submission ready and secrets removed

If time becomes limited, prioritize in this order: ingestion → analytics → LLM query planning → anomaly detection → API → minimal UI → polish.

15. 30-Minute Architecture Walkthrough Preparation

Prepare to explain the following in simple, confident language.

Question: Why did you use Groq?

Answer: Groq provides hosted LLM inference with a free plan and fast responses. I kept the provider configurable and avoided paid dependencies. The LLM is used for language understanding, while Python performs deterministic calculations.

Question: Why not ask the LLM to answer directly from the CSV?

Answer: Direct generation can hallucinate counts and averages. I use the LLM to create a validated query plan, then execute the plan with Pandas. This separates language understanding from computation and makes answers reproducible.

Question: How do you prevent unsafe LLM output?

Answer: The model can only return allow-listed operations and fields. Pydantic validates the JSON, filter values are checked, result limits are capped, and arbitrary Python or SQL is never executed.

Question: How are anomalies detected?

Answer: I use transparent business rules for critical unresolved and aging tickets, plus an IQR rule for unusually long resolution times. Each result includes a reason and threshold so it is explainable.

Question: How would you scale it?

- Replace CSV loading with PostgreSQL or a warehouse.
- Add a semantic layer and query catalog.
- Use background jobs for anomaly refresh.
- Add caching and request tracing.
- Add authentication and role-based access.
- Add evaluation datasets for query-plan accuracy.
- Deploy API and UI separately with monitoring.

Question: What would you improve with more time?

- More robust date interpretation.
- A larger suite of adversarial query tests.
- Database-backed pagination.
- LLM observability and prompt versioning.
- Human feedback on incorrect query plans.
- Configurable anomaly thresholds per business team.

16. Final Submission Checklist

Tick every item before sending the email.

- The repository runs locally from a fresh terminal.
- requirements.txt is complete.
- .env.example exists, but .env is excluded.
- The dataset path is documented.
- GET /health works.
- POST /query works with at least five sample questions.
- GET /anomalies works and includes explanations.
- The UI loads and displays data.
- Groq errors are handled gracefully.
- No paid API or paid service is required.
- README includes architecture, setup, examples, limitations, and screenshots.
- Tests have been run and results are recorded.
- No API key, password, or private file is committed.
- GitHub repository is accessible to the evaluator.
- ZIP opens and contains the complete project.
- Email subject and recipient are correct.

Final mindset:

The evaluator does not need a massive enterprise product. They need evidence that you can translate a business problem into a working AI system, make sound engineering choices, validate LLM output, and explain trade-offs. A smaller system that works reliably is stronger than a beautiful system with broken core logic.

Useful official references

Groq Console: <https://console.groq.com/>

Groq billing FAQ: <https://console.groq.com/docs/billing-faqs>

Groq supported models: <https://console.groq.com/docs/models>